

浅谈基于状态压缩的动态规划

中山纪念中学 姜碧野

QQ: 609295508

E-mail: zqzx06jby@126.com

摘要:

动态规划作为一个高效的算法已经在多个领域得到了广泛应用，而基于状态压缩的动态规划则是一种特殊的动态规划。近年来在信息学竞赛中也涌现出越来越多这种类型的题目。本文将以三道典型例题向你简要介绍基于状态压缩的动态规划的基本方法和适用范围。

关键字:

动态规划，状态的表示，状态压缩

正文

引入:

动态规划作为一种常见的算法，已在信息学竞赛中屡见不鲜，而各种状态的表示作为动态规划的核心内容也早已为各位选手所熟知。但现在经常有这么一种题，它利用我们常见的各种状态表示就会具有后效性，而用搜索又无法满足时间的要求。这时就需要我们对状态的表示进行改进，基于状态压缩的动态规划就出场了！

普通的动态规划一般是以一个个元素作为状态，或是以一整块作为状态，这样虽然直观清楚，但记录的信息太少，因此就会导致后效性的存在。而状态压缩模型则是把许多个元素的状态（比如一行）作为一个整体来表示状态，这样就为下面的计算提供了更多的信息，从而保证了正确性。但是由于是多个元素的集合体，所以要在计算机上储存就要把它压缩，具体的方法将在后面详细描述。

下面，我将由一个简单的例子来引出基于状态压缩的动态规划的基本思路和做法。

问题一：互不侵犯（四川省选 2005）

题目大意：

在 $N \times N$ 的棋盘里面放 K 个国王，使他们互不攻击，共有多少种摆放方案。国王能攻击到它上下左右，以及左上左下右上右下八个方向上附近的各一个格子，共 8 个格子。

输入：两个数 N, K （ $1 \leq N \leq 9, 0 \leq K \leq N * N$ ）

输出：方案数。

样例输入输出：3 2

16

问题分析

由问题很容易联想起经典的“八皇后”问题，似乎就是“皇后”变成了“国王”，而且格子范围似乎也差不多，所求问题也一样。那么这个问题也能用搜索解决吗？

可稍加分析可知搜索是很难胜任的，因为国王的数目可以是很大，加上它与“八皇后”问题的一个本质上的不同便是每个国王只影响周围的一个格子，所以剪枝条件也很少，指数级别的搜索是无法在时限内出解的。

那么一般的动态规划能解决吗？典型的二维 DP $F[I, J]$ 似乎无法很好地把状态表示出来，因此我们只能考虑状态压缩的动态规划。

首先我们要注意到这题的关键——**每个国王只影响周围八个方向的一个格子**，它虽然否定了搜索，却给状态压缩带来了无限生机！

我们改变之前动态规划的思维方式，一行一行地摆放国王，当我们摆放第 I 行时，这一行只会和前后一行的互相影响，而这一行的状态是可以由我们确定的。那是否可以把一行当作一个整体，然后像传统的动态规划那样进行处理呢？让我们试一下。

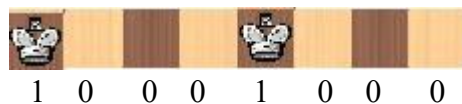
每一行它对下一行的影响就体现在这一行的摆放方式以及之前总共之前放了多少个国王。所以我们可以把**摆放方式**作为状态，这样很容易便得到了一个粗略的方程：

$$F[I, J, S] = \sum F[I-1, K, T]$$

J, K 分别表示一种摆放方式， $F[I, J]$ 表示第 I 行用 J 的摆放方式，且 J 与 K 相兼容。并且 S 等于 T 加上 J 这种方式在这一行放置的国王数。

很明显这个方程是没有后效性的，可关键就在于 J, K 怎么在计算机上表示出来，这就需要我们的主题：**状态压缩**了。

看图便知，每一个格子只有两种状态，放和不放，并且注意到格子宽度最大为 9。由这便想到了熟悉的二进制表示法。每一个格子对应一个二进制位。这样每一行便对应一个 N 位的二进制数，如下图所示：



即十进制的 $128+8=136$

这样我们就可以把一种摆放方式转化为一个数，这样上面方程中的 J, K 就可以用数字来代替。我们就把一行的摆放方式作为**状态**，并把它**压缩**成了一个数！具体的算法流程如下：

对于每一行，我们通过**搜索**得出一个合法状态，然后再搜索上一行与这一行相容的状态再累加即可，状态用 N 位的二进制数表示，最大仅为 512，所以一个 $512*9*81$ 的数组就可以了，还可以用滚动数组的技巧。空间是肯定可以承受的。

而一个粗略的时间复杂度： $O(K*N*2^N*2^N)$ ，似乎大了点。不过注意到由于国王是不能相邻放置的。所以我们可以用一个 $f(n)$ 来表示当列数为 n 时每一行可能的放置总数。则 $f(n) = f(n-1) + f(n-2)$ 初始值： $f(1) = 2, f(2) = 3$ 。则 $f(9) = 89$ 。因此最大也才是 $9*89*89*81 \approx 6000000$ 。是可以承受的。

小结：

至此，这题已得到完美解决。我们可以发现，其实状态压缩的动态规划也具

有普通动态规划的一些基本性质，比如无后效性，而它的核心也是减少重复和消除冗余。只不过它的状态表示方法与平时不同，把一整行的摆放方式作为状态并压缩成数字来储存，这样我们便成功的设计出了动态规划的算法。

思考问题：

1：“八皇后”问题能否用动态规划解决？如果能，时间空间复杂度如何？如果不能，又为什么呢？

2：如果每个国王的影响范围改为上下左右四个方向各 2 格该怎么做？（即NOI2001 炮兵阵地）

现在相信你已对状态压缩的动态规划有所了解，那么我们便来看一个复杂点的例子。

问题 2：Bugs （CEOI2002）

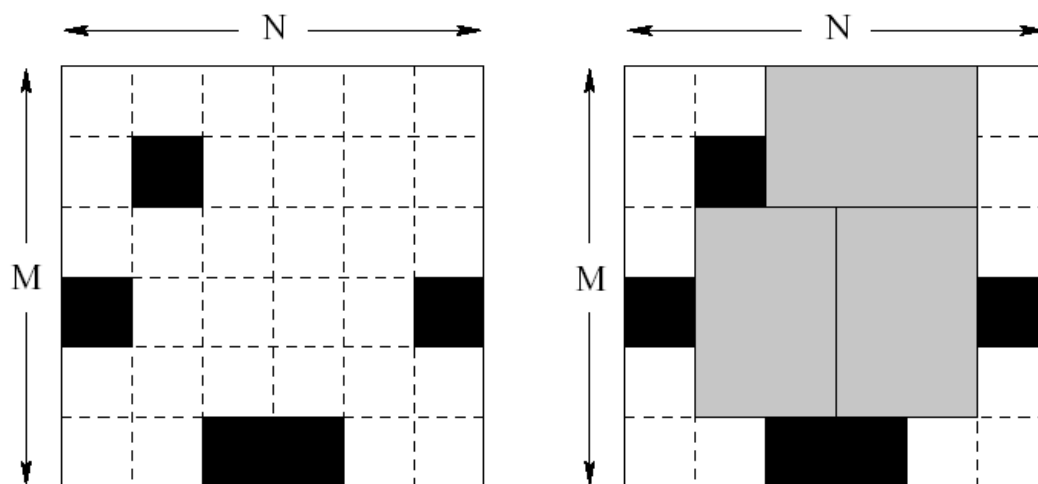
PKU1038

题目大意：对于 $M \times N$ 的矩阵，放置 2×3 的小块，可以旋转，小块之间不可以重叠，矩阵中有一些坏块（ K 个）不可以使用，问最多可以放多少个小块（ $1 \leq N \leq 150, 1 \leq M \leq 10$ ）题目有多组数据。

输入：一个 T ，表示 T 组数据（ $1 \leq T \leq 5$ ）

每组数据先是一行三个数 N, M, K ，接着 K 行，每行2个数 X, Y ，表示 (X, Y) 这个格子损坏。（ X, Y ）在 $[1, 1]$ 到 $[N, M]$ 间。

输出： T 行，每行一个整数，表示对应的数据最多能放多少个小块。



问题分析：

我们比较这题和上题，可以发现有很多类似。都是放置问题，范围都比较小（ $M \leq 10$ ）所以我们考虑是否能像上题那样进行动态规划呢？

如果也用一个10位的二进制数表示每一行的状态，稍加分析便发现不可行。因为是 2×3 的小块。如果竖着放还会影响到前2行的状态，而且每个格子的状态还

要分为是横着放和竖着放。原来的方法不再适用了,怎么办呢?

回想起平时在动态规划中我们遇到状态无法合理表示时经常会用到“加维”的思想,那么在状态压缩中是否也能如此呢?请读者们先仔细思考。下面将直接给出算法

算法设计:

首先,我们依然把一行看作一个整体,看看每一行可以有什么状态以及对周围的有什么影响。假设现在对第X行第Y列进行操作,最简单的状态就是(X,Y)这个位置不放东西,我们记为状态1。否则我们以第X行为底边进行放置,对于每次放置,我们可以竖着或横着放置一个2*3的小块。如果是竖着放,则要求X-2和X-1行的对应位置都必须为空闲,也就是说每一行必须记录一种状态,对于(X,Y),要求(X-1,Y)和(X,Y)都为空闲,我们记为状态2,然后我们还要记录这个格子已经放置,记为状态3。于是我们得到如下状态表示(先不考虑坏的块):

对于(X,Y):

状态1:(X,Y)空闲

状态2:(X,Y)和(X-1,Y)均空闲

状态3:(X,Y)不空闲

那么这三种状态是否就能包括所有情况呢?是否具有后效性呢?我们接着分析。(我们假设在(X,Y)处放置是指以(X,Y)为左下角进行放置)

1: 在(X,Y)处横着放置一块则要求(X-1,Y)(X-1,Y+1)(X-1,Y+2)空闲(不考虑坏的块)那么第X-1层的Y到Y+2为状态1,第X层的Y到Y+2为状态3。

2: 在(X,Y)处竖着放置一块则要求(X-1,Y)(X-1,Y+1)和(X-2,Y)(X-2,Y+1)都为空闲,那么第X-1层的Y到Y+2为状态2,第X层的Y到Y+2为状态3。

3: (X,Y)不放置,这时又有两种可能,一种就是只是(X,Y)为空闲,则(X,Y)为状态1,此时(X-1,Y)可以为任意状态。另一种就是(X,Y)和(X-1,Y)都为空闲,则(X,Y)为状态2,(X-1,Y)为状态1。

由此,我们可以发现,只用这三种状态已经可以很好的表示出所有的情况,这样我们就可以用三进制来表示这三种状态。而 3^M 最多只有 $3^{10}=59049$ 种,是可以承受的。但是像上题那样转移则时间复杂度就是 $O(N*T*3^M*3^M)$,这未免太大了点。不过像上题那样,实际的情况并没有这么多,而且是求最优解,所以我们可以利用搜索中剪枝的思想来对其进行优化。

优化和改进:

对于状态3,注意到小块肯定是一个整体,是不可能分开的,所以状态3必然是连续的2或3个格子。

另外我们要对状态2进行充分分析,状态2对动态规划到底有什么作用。回顾我们设计状态2时是为了解决竖着放置时要求前2行都为空闲的问题,那么如果不是竖着放置,状态2会对它们产生影响吗?经过分析得到以下结论:

如果下一层在对应的位置上没有竖着放置的小块,使用状态2不会比其他解

优。因为假如下一层对应位置不是竖着放置，那如果当前位置改为状态1则不会影响当前解。而状态2还要求上一行为空闲，所以不会更优。

因此状态2必然是和竖放紧密联系的，所以它也必须是连续的2个格子，并且它的下面必须能够竖着放置一个小块，即不是边界和下两行没有坏块。

而状态1就必须我们去一个个搜索了。

另外注意到当前行的状态确定时，上一行的状态 $(X-1, Y)$ 只有当 (X, Y) 为状态1时才需要搜索，而其他情况都是唯一确定的。所以总的枚举量是很少的。远达不到 $3^M \times 3^M$ 级别。

算法流程和上题也差不多，就是在搜索时要注意利用最优性来剪枝优化。还有就是我们之前没有考虑的坏块，只需在搜索时判断当前放置的范围是否有坏块就行了。

虽然经过了剪枝和优化，由于没有理论支持，所以我们仍然怀疑这个算法的实际效果如何。不过实践是检验真理的唯一标准。让我们用数据来检测！

由算法的框架可知，如果坏块越多，则剪枝越多，速度也越快，反之则慢。所以对于这个算法的最坏数据就是 $N=150, M=10, K=0$ ，即没有坏块。这个数据在 Celeron 1.7GHz 512MB 的环境下能够在 0.6s 内出解，因此它的实际效果是非常不错。而在 PKU 的 Online Judge 上提交，不但没有超过 5s 的时限，更是以不到 0.7s 的时间排名第三。可以说的是一个很优秀的算法。

其实这题还有一个复杂度达到了 $O(N \times T \times M \times 3^M)$ 的优秀算法，详见《算法艺术与信息学竞赛》，也可以参考官方解答中的算法。

小结：

这题可以说是一道经典的状态压缩题目。我们在二进制无法很好地表示状态时，沿袭动态规划中“加维”的思想，改用三进制。这样就完整地表示出了状态并成功设计出了动态规划。得出的算法虽然看似复杂度较高，但在转移时又通过仔细分析对搜索进行了较强的剪枝，最终得到了一个另人满意的算法。并在 PKU 上比许多理论复杂度明显占优的算法还要快。这也启示我们在对这种动态规划的状态进行搜索时是可以加入大量剪枝的！

思考问题：

- 1：你能设计出 $O(N \times T \times M \times 3^M)$ 的算法并用上文的思路对其加以优化吗？
- 2：假如没有坏块还需要动态规划吗？为什么？

现在，你应该对状态压缩的动态规划有了较为清楚的认识，想跃跃欲试了吧。那就来挑战最后一道难题吧，它更加复杂，其状态压缩模型也更加隐蔽。

问题3：connect (CEOI2006)

Time limit: 0.5sec

Memory limit : (heap) 32 MB, (stack) 8 MB

问题描述

在 16 位操作系统 DOS 盛行、25*80 显示器还是主流的时代，“Nibbles”是一个相当流行的游戏。然而这个问题，和 Nibbles 没有什么太大的关系。它和一个叫做 “Connect”的游戏有关，这个游戏几乎和 Nibble 完全不同。

Connect 游戏是在一个 R 行 C 列的长方形棋盘上进行的，这里 R 和 C 都是奇数。行和列的编号分别是 1~R 和 1~C。棋盘上的每一个位置有两种状态：

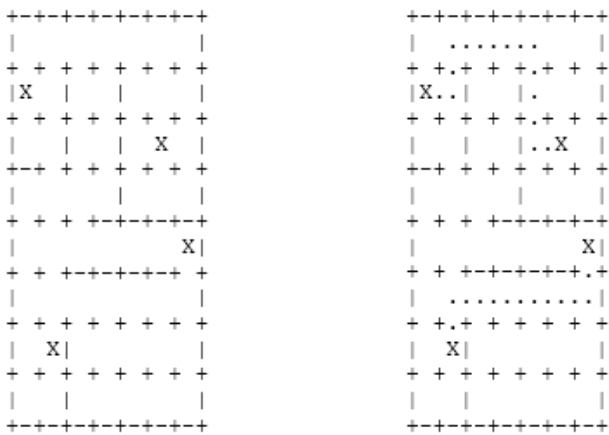
- 1、可以自由通过的；
- 2、被墙壁阻挡的。

而且棋盘还满足以下几条约束：

- 1) 横纵坐标均为偶数的格点被称为房间。在任何情况下，它们都不会被阻挡。
- 2) 横纵坐标均为奇数的格点被称为障碍。它们总是被阻挡的。
- 3) 所有其他的格点被称为走廊，它们可能是被阻挡的，也可能不是。
- 4) 棋盘边缘的走廊总是被阻挡的。

障碍用 ‘+’表示，墙壁有两种，垂直的墙壁用’|’来表示，而水平的墙壁用’-‘来表示（只是为了与’+’相连时美观，因此才会有两种不同的墙壁，两种墙壁起所的作用相同）。房间和可以通过的走廊用空格表示。

游戏开始时有偶数个物品(用’X’表示)放置在棋盘上，每一个都在不同的房间中。物品 A 和 B 之间的路径是由一系列以 A 为起点，以 B 为终点的四个方向的移动组成的，路径包括 A 和 B。路径的长度定义为从 A 到 B 所需要移动的步数，即路径上所有方格数减 1。



游戏者的目标是将所有的物品搭配成对，然后将每一对物品连接起来，组成路径。任何两条路径都不应该相交。游戏结束时，游戏者得到的分数是所有路径长度之和。

试编写一个程序，在给出 Connect 游戏初始条件的情况下，计算出最小可能的分值。

测试数据保证问题有解，但不一定唯一。

输入

输入的第一行包括两个奇数 R 和 C (5≤R≤25, 5≤C≤80)，指定了行数和列数。接下来的 R 行，每行包括 C 个字符，表示棋盘上的每一行。每个字符都是 ‘+’、’|’、’-‘、空格或’X’中的一个，分别表示障碍及两种墙壁、空格表示房间或可通过的走廊，X 表示房间中的物品。

下面是读入整行数据的方法提示（注意 R 和 C 后面的换行要首先读入）：

PASCAL

```
var s : string;  
readln(s);
```

C

```
char s[81];  
gets(s);
```

C++

```
string s;  
getline(cin,s);
```

输出

输出的第一行是一个整数，即最小分值。

接下来的 R 行描述了游戏进行之后棋盘的状态。棋盘的格式应该和输入时一样，而每个路径上的点都应该用'.'来表示。

注意：如果问题有多个解，那么仅需输出一个。

评分标准

如果没有正确地找到最短路径，可以仅给出最小的可能分值，只要这个数字正确，即可得到该数据 80% 的分数。

如果选择仅回答最小分值，则不需要输出其它任何数据。

问题分析：

这是一道较难的题，正式比赛时最高分只有80分，而平均分更是只有13分。题目大意就是把物品两两配对，并使路径总长度最小。这很容易使人想起最优匹配，但这题还有一个重要的限制即路径不能相交，这是匹配所不能力及的，而且25*80的规模也不是匹配所能承受的。除此之外，也很容易想到最短路径，但对于保证路径不交叉似乎很难做到。不过广东韶关一中的刘家骅同学在网上同步赛中用最短路+随机调整的算法拿到了75分的高分，有兴趣的读者可以与他联系。而盲目的搜索也不可能在时限内出解。另外要注意到这题的时限卡得很紧，只有0.5s, 因此我们必须设计出高效的算法,自然就会想到动态规划了！

有了前两题的经验,聪明的读者应该一眼就看出普通的动态规划肯定是不行的，因为路径错综复杂，还可以兜一个大圈，因此普通的动态规划肯定是有后效性的。

那么状态压缩的动态规划能否满足要求呢？我们按照前面两题的思路，如果是一行一行的进行递推，但由于每行的格子之间也可以连接路径，这样做似乎会很复杂。还有就是 $N \leq 25$,就算是用二进制编码， 2^{25} 也是无法承受的，更何况每个格子的状态更是多种多样。怎么办呢？.....只能继续分析。

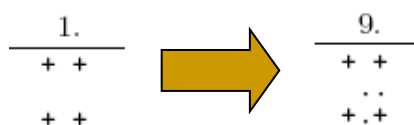
首先注意到当坐标全为奇数时必是障碍物，全为偶数时为房间，而且物品全在房间内。那么是否可以利用这些条件来减少空间。看一下图后发现，路径是从房间走到房间的，中间经过了房间和走廊。而走廊的作用仅仅就是可以将两个房间隔开。所以实际上我们可以只考虑房间，而障碍物对我们也没有影响，只需考虑走廊是否把房间隔开。这样规模就变成12*39！可以考虑动态规划了。

算法设计：

首先分析每个房间可能的状态。

下图1所示的为迷宫中的一个“房间”以及与其相邻的四个走廊可能的11种状态。

四周走廊都没有路径的空房间，事实上也有可能在将来的某个时刻成为路径的一部分，即下图所示过程：



因为路径是可能兜圈的。

最终时间复杂度： $O(R \cdot C \cdot 2^{(R+1)/2} \cdot 11)$

小结：

这题的确是一个比较麻烦的难题。相对前两题，它的模型更加隐蔽，很容易就会去想其他算法。其初始规模也不适合直接动态规划，而正是逐步的分析才使状态压缩模型逐渐显露出来。另外上文所说的“插头法”也确实是一个漂亮的方法，它紧紧地抓住了状态的本质，即只有路径才对后面的决策有影响。这才使状态压缩得以实现。

思考问题：

1：此题能否像前两题那样逐行递推？如果能，它的时间和编程复杂度如何？如果不能，又为什么呢？

**2：如果仅有两对物品，是否有更快更方便的解法？

总结：

状态压缩的动态规划的本质和一般的动态规划是相同的，而它的最大特点就是它的状态不再是孤立的一个元素，而是多个元素的集合体。这样就能为接下来的决策提供更多信息。另外我们要注意到这种动态规划的状态大多都是指数级别的！所以我们必须对状态进行合理的优化。上文主要讲了两种方法：

1:在对状态进行搜索时注意剪枝的应用，尽可能地减少无用状态。

2:抓住状态的本质，只储存对后面的决策真正有用的信息。

相信你们已经掌握了状态压缩的动态规划的基本方法。但这类动态规划本身就非常灵活多变，本文只是讲述了其中的一点皮毛，希望能对大家起抛砖引玉的作用。而在实际应用中，只有仔细分析并结合其他技巧才能使状态压缩的动态规划在信息学这片广阔的天空中飞得更远。

参考文献：

1：《算法艺术和信息学竞赛》刘汝佳，黄亮著

2：Winter Camp2007《CEOI2006题目讨论》PPT 王宏